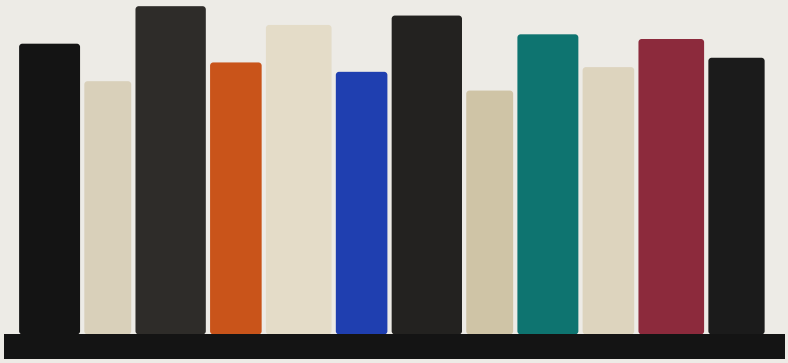
SwaraVeda

SWARAVEDA

A community archive for India's endangered dialects, with Vani, its speech and language AI.

Vani: Preserving Voices, Empowering Generations



Live site: https://swara-veda.vercel.app
API: https://swaraveda-api.onrender.com
Code: github.com/Mohamed-Ali-4/SwaraVeda

CONTENTS

What this guide covers

1. Introduction . . . 3
2. The project at a glance . . . 5
3. System architecture and technology . . . 7
4. Every feature, and how it works . . . 9
5. Database design . . . 16
6. API reference . . . 17
7. Vani: the AI and NLP explained . . . 18
8. Deployment, configuration and DevOps . . . 20
9. Testing and quality . . . 21
10. Challenges we faced and how we solved them . . . 22
11. Limitations and future work . . . 23
12. Viva questions and model answers . . . 24
13. Glossary . . . 29

How to use this guide
Chapters 1–3 give the story of the project: the problem, the idea and how the system fits together. Chapter 4 walks through every feature: what it does, how it works step by step, and which technology does the work. Chapters 5–10 cover the database, API, AI, deployment, testing and the problems solved along the way. Chapter 12 is a bank of likely viva questions with model answers.

WHY SWARAVEDA EXISTS

1. Introduction

1.1 The problem

India has 22 scheduled languages, but people actually speak hundreds of languages and thousands of dialects. Many of them, such as Toda, Tulu, Juang, Koya, Paniya, Toto and Kutchi, are spoken mostly by small and ageing communities and have little or no written record. When the last fluent speakers are gone, the songs, stories, proverbs and everyday talk carried by those dialects disappear with them. UNESCO lists a large number of Indian languages as vulnerable or endangered.

Today's speech technology (voice assistants, captioning, translation) is built only for widely spoken languages, because it needs large amounts of transcribed audio. Endangered dialects have no such data on the internet, so they are left out twice: first by the loss of speakers, then by technology.

1.2 Our solution

SwaraVeda (Sanskrit swara, voice or note, and veda, knowledge) is a web application where communities can record, transcribe, correct, translate and preserve speech in their own dialects. Its AI assistant, **Vani** (Sanskrit for speech), produces a first-draft transcript and English translation of every recording. Native speakers then correct the draft and verifiers approve it. Every approved passage becomes part of an open, consent-based dataset that can later be used to train speech AI for these dialects.

> **The idea in one line**
> Record a voice, let AI draft the words, let the community make them right, and build the dataset that endangered dialects have never had.

1.3 Objectives

- Give anyone with a phone or laptop a simple way to record or upload speech in their dialect, even without an internet connection.
- Use AI (speech recognition and machine translation) to cut the manual effort of transcription.
- Keep humans in charge of quality: native speakers correct and verifiers approve each passage.
- Respect speakers: collect separate consent for public listening and for AI training, and keep private recordings private.
- Make the archive searchable by exact words and by meaning, with playback from the exact moment a phrase is spoken.
- Produce a clean, exportable training dataset (audio and timestamps, AI draft and human correction) for future model training.
- Present the 139 catalogued dialects in a way that invites people in, through the interactive dialect shelf.

1.4 Scope

In scope	Out of scope (future work)
Web app for desktop and phone; recording, upload, offline queue	Native mobile apps (the web app works on phones and installs offline assets)
AI transcription, translation, keyword and meaning search	Training our own dialect-specific speech model (planned; dataset export is ready)
Correction and verification workflow, roles, dataset export	Speaker diarisation (who spoke when), audio denoising beyond filters
139 dialects catalogued with region, script and speech-recognition hint	Automatic language identification

1.5 Who uses it

Role	Who	Can do
Contributor	Any signed-up user (default role)	Record and upload, correct transcripts, translate, tag, see public recordings and their own private ones
Verifier	Trusted native speakers or linguists	Everything above, plus approve corrected passages, see private recordings, add reference recordings, export the dataset
Admin	Project owners (set by ADMIN_EMAILS)	Everything above, plus change people's roles, create, edit and delete dialects

IF YOU ONLY REMEMBER ONE PAGE

2. The project at a glance

2.1 The 30-second answer

When asked “What is your project?”
“SwaraVeda is a community archive for endangered Indian dialects. People record speech in the browser, even offline. Our AI, Vani, transcribes it with OpenAI's Whisper model and translates it to English with a large language model, both served by Groq. Native speakers correct the transcript line by line while listening, and verifiers approve it. Everything is searchable by exact words through PostgreSQL full-text search and by meaning through sentence embeddings and pgvector. Verified, consented passages can be exported as a training dataset, so the archive itself becomes the data needed to build speech AI for dialects that don't exist on the internet. It's built with React and FastAPI on Supabase Postgres, and deployed on Vercel and Render.”

2.2 Key numbers

Item	Value
Dialects catalogued	139 (each with region, script, ISO code where known and a speech-recognition language hint)
Frontend pages	Login, Home, Recordings, New recording, Recording detail, Search, Dialect shelf, Dialect list, Review, Profile, Manage
API endpoints	About 45 REST endpoints across 12 routers
Database tables	7: contributors, dialects, recordings, segments, tags, recording_tags, jobs
Automated backend tests	111 passing (SQLite, plus real PostgreSQL in CI)
AI models	Whisper large-v3 (speech to text), a Groq-hosted LLM (translation; automatic model fallback), multilingual MiniLM (384-dimension embeddings)
Audio standard	16 kHz mono Opus at 32 kbps after cleaning (high-pass 80 Hz, loudness normalisation to −18 LUFS)
Hosting	Vercel (frontend), Render (API), Supabase (PostgreSQL + file storage)
Cost	Runs on free tiers; AI is called over an API, so no GPU server is needed

2.3 The journey of one recording

Step	What happens	Done by
1. Record	Speaker is recorded in the browser (or a file is chosen); title, dialect, speaker, genre, place, date are filled in; two consents are asked	Contributor + browser MediaRecorder
2. Upload	Browser asks the API for a signed upload URL and sends the audio straight to Supabase Storage; offline, it's queued in IndexedDB	React app, FastAPI, Supabase
3. Clean	Server downloads the audio and uses ffmpeg to convert it to 16 kHz mono, filter rumble and even out loudness	FastAPI background task + ffmpeg
4. Transcribe	Whisper large-v3 turns speech into timed segments, with a language hint from the dialect (e.g. Tulu → Kannada)	Vani (Groq API)
5. Index	Each segment's text is turned into a 384-number vector for meaning search	Hugging Face MiniLM + pgvector
6. Correct	Native speaker listens segment by segment and fixes the text; can ask Vani for an English translation and transliteration	Contributor
7. Verify	A verifier approves each corrected segment; when all are approved, the recording becomes “verified”	Verifier
8. Use	Searchable by words and meaning, playable from the exact second, and exportable as training data (if AI consent was given)	Everyone / verifiers

HOW THE PIECES FIT

3. System architecture and technology

3.1 Architecture

SwaraVeda is a three-tier web application: a single-page React frontend, a stateless FastAPI backend, and managed data services. Heavy AI work is delegated to hosted model APIs, so the backend can run on a free server.

Browser (phone / laptop)
React 19 + Vite + Tailwind · MediaRecorder · IndexedDB · Service worker

↓ HTTPS / JSON with JWT · ↓ audio goes directly to storage with a signed URL

FastAPI backend on Render
Auth · Recordings · Segments · Review · Search · Export · Background transcription jobs · ffmpeg

↓ SQL ↓ Storage API ↓ AI APIs

Supabase PostgreSQL	Supabase Storage	Groq API	Hugging Face
7 tables · full-text search · pgvector	Bucket “recordings” · signed upload URLs	Whisper large-v3 · LLM translation	Multilingual MiniLM embeddings

Google Identity	Wikimedia Commons	GitHub Actions
“Continue with Google” sign-in	Openly licensed reference audio	Tests + build on every change

3.2 Technology stack, and why each was chosen

Layer	Technology	Why we chose it
UI framework	React 19 + TypeScript 6	Component model for a rich interactive app; TypeScript catches errors at build time
Build tool	Vite 8	Very fast dev server and optimised production bundles
Styling	Tailwind CSS v4 + custom CSS	Design tokens (paper, ink, cobalt) in one place; custom CSS for the 3D shelf and animations
Routing	React Router 7	Client-side pages, protected routes for signed-in and admin-only pages
HTTP client	Axios	Interceptors attach the login token and handle expired sessions; upload progress events
Icons / fonts	react-icons; Newsreader, Inter Tight, Noto Serif & Noto Sans for 12 Indian scripts	Correct rendering of Devanagari, Tamil, Telugu, Kannada, Malayalam, Bengali, Gujarati, Odia, Gurmukhi, Ol Chiki, Meetei Mayek, Tibetan, Lepcha, Limbu
Backend framework	FastAPI (Python)	Fast, async-capable, automatic request validation and interactive API docs (/docs)
Data models / ORM	SQLModel (SQLAlchemy + Pydantic)	One class is both the database table and the validation schema
Database	PostgreSQL on Supabase	Reliable relational store with built-in full-text search and the pgvector extension
Vector search	pgvector	Stores 384-dimension embeddings and finds the nearest by cosine distance inside Postgres
File storage	Supabase Storage	Cheap object storage; signed URLs let the browser upload directly without passing through our server
Authentication	JWT (python-jose), bcrypt (passlib), Google Identity Services	Stateless login tokens; passwords stored only as salted hashes; one-tap Google sign-in
Speech to text	Whisper large-v3 via Groq	State-of-the-art multilingual ASR (~100 languages); Groq runs it very fast with no GPU of our own
Translation	LLM via Groq (Llama family; automatic fallback)	Understands noisy, dialect-mixed text; returns JSON with translation and transliteration
Embeddings	paraphrase-multilingual-MiniLM-L12-v2 via Hugging Face	Small multilingual sentence model (50+ languages) for meaning search
Audio processing	ffmpeg / ffprobe	Convert any format to clean 16 kHz mono speech audio and measure duration
Offline	IndexedDB + Service Worker	Recordings are saved on the device and sent later; the app shell loads without a network
Testing	pytest, Playwright, oxlint	Unit/API tests, real-browser end-to-end checks, fast linting
CI / hosting	GitHub Actions, Vercel, Render	Every change is tested; frontend and API deploy automatically from the main branch

THE CORE OF THE REVIEW

4. Every feature, and how it works

Each feature below lists what the user sees, what happens behind the scenes step by step, and the technology and code involved.

4.1 Sign-in, sign-up and accounts

What it does. People create an account with name, email, mobile number and password, or use “Continue with Google”. They can sign in later with **either** their email or their mobile number.

How it works

- **Sign-up:** the API checks the email and phone are unused, hashes the password with **bcrypt** (salted, slow by design), stores the contributor and returns a **JWT** (JSON Web Token) signed with the server's secret (HS256), valid for 7 days.
- **Mobile numbers** are normalised to the international E.164 format, so “98765 43210”, “+91-98765-43210” and “919876543210” all match the same account.
- **Sign-in** accepts an email or a phone number in one field, verifies the bcrypt hash and returns a new token.
- **Google sign-in:** the browser shows Google's button (Google Identity Services). Google returns a signed ID token; the API verifies its RS256 signature against Google's public keys, checks the audience (our client ID) and issuer, then finds or creates the account.
- The React app stores the token and sends it as `Authorization: Bearer …` on every request; when it expires the user is signed out gracefully. A /auth/refresh endpoint issues a fresh token.
- **Admins** are whoever's email is listed in the server's ADMIN_EMAILS setting; they're promoted automatically when they sign in.
- **Profile page:** change display name, phone and native dialect; change password (needs the current password).

Technology used	Where in the code
FastAPI, python-jose (JWT), passlib + bcrypt, Google Identity Services, React context (useAuth)	backend/app/auth.py, google_auth.py, phone.py, routers/auth_routes.py; frontend/src/lib/auth.tsx, pages/Profile.tsx

4.2 The login page and reactive logo

What it does. The first screen visitors see: the SwaraVeda logo (written “Swaraveda” in Devanagari) cycles through ten Indian scripts every second, beside a shelf of dialect books and the sign-in card.

How it works

- The word Swara is shown in English, Hindi, Tamil, Telugu, Kannada, Malayalam, Bengali, Gujarati, Odia and Punjabi, changing every **1 second** (a React hook with a timer). Only that word changes colour, into each script's own ink; “Veda” (in Devanagari) and SWARAVEDA stay cobalt.
- The change is a smooth cross-fade (Web Animations API): the old word blurs and lifts away while the new one settles in, and the slot eases to the new word's width, which is measured in advance, so words never overlap.
- A row of ten book spines, one per script and labelled with a dialect written in it (Toda, Tulu, Juang…), mirrors the logo: the matching book lifts out in the same ink. Clicking a book or the logo jumps to that script. On phones the shelf scrolls to keep it in view.

- Background sound-wave lines (an HTML canvas) swell around the mouse pointer and take the current script's colour.
- Fonts are loaded with only the letters the logo needs (Google Fonts “text=” subsetting), so the page stays light.
- Respects “reduce motion” accessibility settings.

Technology used	Where in the code
React hooks, Web Animations API, CSS transitions, HTML Canvas, Google Fonts (Noto Serif for each script)	frontend/src/pages/Cover.tsx, Cover.css, components/ReactiveLogo.tsx, VoiceField.tsx, lib/swara.ts

4.3 Home dashboard

What it does. A welcome page showing the archive in numbers (speech recorded, recordings, passages verified, dialects), the latest recordings and a mini dialect shelf.

How it works

- Calls GET /stats, which counts recordings, verified recordings, segments, verified segments, dialects, contributors and total audio seconds.
- Durations are shown in seconds, minutes or hours depending on size.
- Shows recent recordings as cards with dialect, genre, duration and status (uploaded, transcribing, transcribed, verified).
- Uploads waiting on the device (offline queue) are shown with a “send now” control.

Technology used	Where in the code
React, Axios, FastAPI aggregate SQL queries	frontend/src/pages/Dashboard.tsx; backend/app/routers/stats.py

4.4 Recording or uploading audio (3-step wizard)

What it does. A guided form: **1. Audio** (record now or choose a file), **2. About the recording** (title, dialect, speaker, age group, genre, place, date, notes), **3. Consent & upload**.

How it works

- Recording uses the browser's **MediaRecorder** API with getUserMedia. Echo cancellation and noise suppression are switched **off** (they distort natural speech and singing); auto gain stays on. The best supported format is picked: WebM/Opus, then MP4, then Ogg.
- A live timer and input-level meter (Web Audio AnalyserNode) show it's working; the take can be played back and re-recorded before continuing.
- **Two separate consents**: “anyone may listen” (public) and “may be used to train speech AI”. Both are off by default.
- **Upload:** the browser asks POST /recordings/upload-url; the API creates a **signed upload URL** in Supabase Storage under the user's own folder. The browser PUTs the file directly to storage, with a progress bar, then POSTs the details to /recordings.
- Optionally starts transcription immediately (POST /recordings/{id}/transcribe).
- Genre is one of conversation, narrative (stories), song, prayer, proverb or other, so the archive can be browsed by type.

Technology used	Where in the code
MediaRecorder, getUserMedia, Web Audio API, Axios upload progress, Supabase signed URLs, FastAPI	frontend/src/pages/NewRecording.tsx, lib/offline.ts (sendRecording); backend/app/routers/recordings.py, storage.py

4.5 Offline recording

What it does. In villages with poor connectivity, recordings are never lost: they're saved on the device and uploaded automatically when the connection returns.

How it works

- If an upload fails because there's no connection (the browser is offline, or the request got no response), the audio Blob and all its details are saved in the browser's **IndexedDB** database (store “pending-uploads”).
- When the browser fires the “online” event, or when the user presses “send now”, the queue is replayed in order: upload, save details, start transcription. It stops at the first failure to try again later, and each sent item is removed.
- A **service worker** caches the app shell (HTML, JavaScript, CSS, fonts), so the app opens even with no network.
- The header shows a badge with the number of recordings waiting.

Technology used	Where in the code
IndexedDB, Service Worker + Cache API, online/offline events	frontend/src/lib/offline.ts, public/sw.js, components/Layout.tsx (PendingUploads)

4.6 Vani: automatic transcription

What it does. Turns the audio into a timed transcript, split into short segments (sentences or phrases) with start and end times.

How it works

- POST /recordings/{id}/transcribe creates a **Job** row (status queued) and sets the recording to “transcribing”. The work runs as a FastAPI **background task**, so the user isn't kept waiting. If a job is already running it is returned instead of starting another.
- **Download:** the audio is streamed to a temporary folder (size-limited to 200 MB), with a User-Agent so archives like Wikimedia accept it.
- **Clean:** ffmpeg converts it to mono, 16 kHz (what Whisper expects), applies a high-pass filter at 80 Hz (removes rumble and handling noise) and loudness normalisation (EBU R128, −18 LUFS) so quiet speakers become clear, then compresses to Opus 32 kbps (must be under 25 MB). ffprobe measures the duration.
- **Recognise:** the file is sent to **Whisper large-v3** on Groq with **verbose_json** output (segment timestamps) and temperature 0 (most likely text). Each dialect carries an **ASR language hint**, the closest major language Whisper knows (e.g. Tulu → Kannada, Bhojpuri → Hindi, Toda → Tamil), which sharply improves accuracy.
- Segments are saved (seq, start_ms, end_ms, asr_text). Progress is reported (5% → 25% → 70% → 85% → 100%) and the page shows it live by polling GET /jobs/{id}.
- **Embeddings** are computed for every segment for meaning search; if that service is unavailable, the transcript is still saved.
- Errors (download failed, file too large, AI service error) are saved on the job and shown to the user in plain language.

Technology used	Where in the code
FastAPI BackgroundTasks, httpx streaming, ffmpeg, Groq Whisper large-v3, Hugging Face embeddings	backend/app/pipeline.py, audio.py, nlp.py; frontend/src/pages/RecordingDetail.tsx (JobBanner)

4.7 The recording page: player, synced transcript and correction

What it does. Where a recording is listened to, read and corrected.

How it works

- A **waveform player** draws the audio's shape; click anywhere to jump, with play/pause and playback-speed controls.
- The **transcript is synchronised**: the segment being spoken is highlighted as the audio plays, and clicking a segment plays from its start time.
- Each segment shows the AI draft and an editor for the corrected text, transliteration and English translation. Saving sends PATCH /segments/{id}.
- **Translate with Vani** fills in an English translation and a Latin-script transliteration for a segment (see 4.8).
- Owners and admins can edit details (title, dialect, speaker, genre, place, date, consents) or delete the recording.
- Tags (themes such as harvest, wedding, lullaby) can be added and removed; they're shared across the archive.
- Private recordings return 404 to anyone not allowed to see them, so their existence isn't revealed.

Technology used	Where in the code
HTML5 Audio, Canvas waveform, React state, REST PATCH	frontend/src/pages/RecordingDetail.tsx, components/Waveform.tsx, SegmentItem.tsx, RecordingTags.tsx; backend/app/routers/segments.py, tags.py

4.8 Translation and transliteration

What it does. Gives an English meaning and a Roman-letter transliteration for each passage, so researchers and younger generations can understand it.

How it works

- POST /segments/{id}/translate sends the corrected text (or the AI draft) to a large language model on Groq with a system prompt explaining that the text may be a dialect of a named language, may contain transcription errors, and that it must reply with JSON: {translation_en, transliteration}.
- JSON mode and a low temperature (0.2) keep answers consistent and machine-readable; the result is offered as a suggestion that a person accepts or edits.
- **Resilience:** AI providers retire models. If the configured model no longer exists, the API asks Groq which models the key can use, picks the best chat model (skipping speech, safety and TTS models), remembers it and retries automatically. We added this after the original model was retired.

Technology used	Where in the code
Groq chat completions (LLM), JSON mode, prompt engineering	backend/app/nlp.py (translate_text, _chat, _pick_chat_model); routers/segments.py

4.9 Review and verification workflow

What it does. Quality control: every passage moves through three states before it counts as verified.

How it works

- **Needs correction**: has an AI draft but no human correction yet.
- **Needs verification**: a person has corrected it but no verifier has approved it.
- **Verified**: approved by a verifier (verified_by and verified_at are recorded).
- The Review page (GET /review/segments?state=…) lists passages in each state, filterable by dialect, each with a play button for just that clip.
- Only verifiers and admins can approve (POST /segments/{id}/verify); a passage with no correction can't be approved. When the last passage of a recording is approved, the recording's status becomes “verified”.
- This is a **human-in-the-loop** design: AI does the slow first draft, and humans guarantee accuracy.

Technology used	Where in the code
FastAPI role dependencies (require_verifier), SQL filters	backend/app/routers/review.py, segments.py, auth.py; frontend/src/pages/Review.tsx

4.10 Search: by exact words and by meaning

What it does. Find any phrase across the archive and jump straight to the moment it is spoken.

How it works

- **Exact-word search** (GET /search?q=…) uses PostgreSQL full-text search: to_tsvector and plainto_tsquery with the **“simple”** configuration (no English stemming, which would damage Indian-language words), ranked by ts_rank. It searches corrected text first, falling back to the AI draft. On SQLite in tests it falls back to LIKE.
- **Meaning search** (GET /search/semantic) turns the query into a 384-number vector with the same multilingual MiniLM model, then pgvector finds the segments whose vectors are closest by **cosine distance** (the <=> operator), so “rain” can find a passage about the monsoon.
- Results show the matched text highlighted, the recording, the dialect and a play button that starts at the segment's start time.
- Only recordings visible to the user are searched (privacy rules apply to search too).
- Meaning search is currently paused because the Hugging Face free credits are used up; the API returns a friendly message and exact-word search keeps working.

Technology used	Where in the code
PostgreSQL full-text search, pgvector, Sentence embeddings (MiniLM), React	backend/app/routers/search.py, pipeline.py (embed_segments), nlp.py (embed_texts); frontend/src/pages/Search.tsx

4.11 The dialect shelf (VHS-tape interface) and dialect list

What it does. The 139 dialects are presented as a 3D shelf of video tapes, one per dialect, each with generated cover art, plus a searchable list view.

How it works

- Each tape's spine shows the dialect's name in Latin and native script. The tapes are drawn with CSS 3D transforms and generated artwork (canvas), so every dialect has its own distinctive cover.
- Hovering tilts and pulls a tape out; clicking slides it open to a detail panel with the dialect's region, script, ISO 639-3 code, endangerment notes and description.

- **Voices**: the panel lists recordings in that dialect. “Listen” plays the first one; each can be played or paused, and there's a link to all recordings.
- **Record** opens the wizard with the dialect already selected.
- **Hear it elsewhere**: links to YouTube folk-song searches, Wikimedia Commons and the Internet Archive, for dialects that don't have recordings yet.
- The list view (/dialects/list) shows each dialect with a danger scale; admins can add or edit dialects and set the speech-recognition hint.
- Performance: tape artwork is drawn once and cached, so the shelf stays smooth on phones.

Technology used	Where in the code
CSS 3D transforms, Canvas, React, REST	frontend/src/pages/DialectShelf.tsx/.css, components/TapeArt.tsx, lib/dialectInfo.ts, pages/Dialects.tsx; backend/app/routers/dialects.py, data/dialects.json

4.12 Reference recordings from Wikimedia Commons

What it does. Verifiers can add openly licensed recordings made by others, such as Lingua Libre pronunciations or field recordings, so a dialect has something to listen to from day one.

How it works

- Manage → Add a reference recording → **Find on Wikimedia Commons** searches Commons (and Lingua Libre by the dialect's ISO code) through its public API.
- Only files under open licences are offered: public domain, CC0, CC BY and CC BY-SA. Each result shows its licence and author, which are kept with the recording for credit.
- A verifier can also paste any audio link with its licence and credit. The link must be http(s).
- Reference recordings play on the shelf and can be transcribed like any other. Only openly licensed ones count toward training data.

Technology used	Where in the code
Wikimedia Commons API (MediaWiki), httpx with a descriptive User-Agent	backend/app/routers/references.py; frontend/src/pages/Manage.tsx (ReferenceRecording)

4.13 Training-dataset export

What it does. Downloads the corrected archive as a dataset for training or evaluating speech AI.

How it works

- GET /export/segments?format=jsonl|csv (verifiers and admins only). Optional filters: dialect, and verified passages only (the default in the app).
- **Only** recordings whose speakers consented to AI training are included, and only passages with a human correction.
- Each row has segment and recording IDs, title, dialect, audio URL, start_ms and end_ms, the AI draft (asr_text), the human correction, transliteration, translation, speaker label, verified flag and licence. Pairing the AI draft with the correction lets us measure word error rate and fine-tune models.
- The file streams, so large exports don't use much memory. It's named swaraveda-dataset-YYYY-MM-DD.jsonl or .csv.

Technology used	Where in the code
FastAPI StreamingResponse, JSON Lines, CSV	backend/app/routers/export.py; frontend/src/pages/Manage.tsx (DatasetExport)

4.14 Managing people and roles

What it does. Admins decide who can approve transcripts.

How it works

- Manage → People lists users (GET /admin/users) with their role; an admin can switch someone between contributor, verifier and admin (PATCH /admin/users/{id}).
- An admin can't remove their own admin role by accident.
- Server-side checks (require_verifier, require_admin) protect every sensitive endpoint, so hiding a button is never the only protection.

Technology used	Where in the code
FastAPI dependencies, role-based access control (RBAC)	backend/app/routers/admin.py, auth.py; frontend/src/pages/Manage.tsx (Users)

4.15 Privacy, consent and security, built in

Concern	How SwaraVeda handles it
Speaker consent	Two separate, explicit choices per recording: public listening, and AI training. Both default to no.
Private recordings	Visible only to the uploader, verifiers and admins; everyone else gets 404, in lists, detail pages, transcripts and search alike.
Passwords	Never stored, only bcrypt hashes. Minimum 8 characters. Changing a password needs the current one.
Sessions	Signed JWTs with an expiry; the signing secret lives only on the server.
Uploads	Short-lived signed URLs, scoped to the user's own folder; the storage service key never reaches the browser.
Secrets	All API keys live in Render's environment settings, never in the code or GitHub.
Input validation	Pydantic validates every request (types, lengths, email format, phone format, http(s)-only audio links).
CORS	The API only accepts browser requests from the configured site origins.
Licensing	Each recording carries a licence (default CC BY-NC 4.0); references keep their original open licence and credit.

Why privacy is a feature, not an afterthought
These are often intimate cultural recordings: rituals, family songs, the voices of elders. Communities will only contribute if they trust the archive, so consent is asked per recording, defaults to private, and is enforced on the server for every query.

POSTGRESQL ON SUPABASE

5. Database design

Seven tables. Primary keys are UUIDs (safe to expose in URLs and unguessable). Timestamps are stored in UTC.

Table	Purpose	Important columns
contributors	User accounts	id, display_name, email (unique), phone (unique, E.164), password_hash, role (contributor/verifier/admin), native_dialect, created_at
dialects	The catalogue of dialects	id, name, iso_code (ISO 639-3), region, script, endangerment_status, description, asr_language (hint for Whisper)
recordings	One audio recording	id, title, audio_url, storage_path, duration_seconds, dialect_id → dialects, contributor_id → contributors, speaker_name, speaker_age_group, genre, recorded_place, recorded_at, consent_public, consent_ai_training, licence, status, notes
segments	A timed passage of a transcript	id, recording_id → recordings, seq, start_ms, end_ms, asr_text (AI draft), corrected_text, transliteration, translation_en, speaker_label, verified_by → contributors, verified_at, embedding vector(384)
tags	Shared themes	id, label (unique), category
recording_tags	Many-to-many link	recording_id + tag_id (composite primary key)
jobs	Background AI work	id, recording_id, kind (transcribe), status (queued/running/completed/failed), progress 0–100, error, created_at, updated_at

Relationships

- One dialect has many recordings; one contributor uploads many recordings; one recording has many segments, many jobs and many tags.
- Segments keep **both** the AI's text and the human's text; this pair is what makes the training dataset valuable.
- Recording status lifecycle: **uploaded → transcribing → transcribed → verified**.
- Startup migrations add new columns and the vector extension safely on existing databases; a /health/db endpoint checks every table, a test write and storage.

FASTAPI · INTERACTIVE DOCS AT /DOCS

6. API reference

Area	Endpoints	Who
Auth	POST /auth/register, /auth/login, /auth/google, /auth/refresh, /auth/change-password · GET /auth/config, /auth/me · PATCH /auth/me	Public / signed in
Recordings	POST /recordings/upload-url, /recordings · GET /recordings (filters: dialect, genre, status, q, paging), /recordings/{id}, /recordings/{id}/segments · PATCH, DELETE /recordings/{id} · POST /recordings/{id}/transcribe	Signed in; owner/admin to edit or delete
Segments	GET, PATCH, DELETE /segments/{id} · POST /segments/{id}/translate · POST /segments/{id}/verify	Signed in; verify needs verifier
Review	GET /review/segments?state=needs_correction|needs_verification|verified	Signed in
Search	GET /search?q= · GET /search/semantic?q=	Anyone (respects privacy)
Dialects	GET /dialects, /dialects/{id} · POST, PATCH, DELETE /dialects	Read: anyone; write: admin
Tags	GET, POST /tags · GET, DELETE /tags/{id} · GET, POST /recordings/{id}/tags · DELETE /recordings/{id}/tags/{tag_id}	Signed in
Jobs	GET /jobs · GET /jobs/{id}	Signed in
References	GET /references/commons	Verifier
Export	GET /export/segments?format=jsonl|csv	Verifier
Admin	GET /admin/users · PATCH /admin/users/{id}	Admin
Health	GET /health · GET /health/db · GET /stats	Anyone

SPEECH AND LANGUAGE TECHNOLOGY

7. Vani: the AI and NLP explained

7.1 Where AI is used

Task	NLP / AI technique	Model	Output
Clean audio	Signal processing: resampling, high-pass filter, loudness normalisation	ffmpeg (not AI)	16 kHz mono speech audio
Speech to text	Automatic speech recognition (ASR), an encoder–decoder Transformer trained on 5M+ hours	Whisper large-v3 (1.55B parameters)	Timed text segments
Translation	Neural machine translation with a large language model, prompted for JSON	Groq-hosted LLM (Llama family, auto-selected)	English translation + transliteration
Meaning search	Sentence embeddings + vector similarity (semantic search)	multilingual MiniLM-L12 (384-d)	Ranked passages by meaning
Word search	Information retrieval: tokenising, inverted index, ranking	PostgreSQL full-text search	Ranked passages by words

7.2 Key ideas in simple words

- **Whisper** listens to 30-second windows of audio as a spectrogram (a picture of sound frequencies over time). An encoder turns it into numbers, and a decoder writes the text word by word, also predicting timestamps. It was trained on 680,000 hours originally, and large-v3 on far more, across about 100 languages.
- **Language hint:** Whisper doesn't know Tulu, but it knows Kannada, which shares the script and much vocabulary. Telling it “expect Kannada” gives a far better first draft than letting it guess. Each dialect stores its closest supported language.
- **Embeddings** place sentences as points in a 384-dimension space where similar meanings sit close together. Cosine distance measures the angle between two points.
- **pgvector** adds a vector column type and distance operators to PostgreSQL, so meaning search runs inside the same database, with no separate vector database.
- **LLM translation** works even with spelling errors and mixed languages, because the model uses context. We ask for JSON so the app can read the answer reliably.
- **Human in the loop:** the AI's output is always a draft; people correct and approve it. The corrections then become training data that improve the AI.

7.3 Honest limitations

- Whisper has never been trained on most of these dialects, so first drafts can have many errors. That is exactly why correction exists and why we collect a dataset.
- Accuracy will be measured with **Word Error Rate (WER)**: (substitutions + deletions + insertions) ÷ number of words in the correct transcript.
- Hosted AI depends on outside services and their free limits (e.g. the Hugging Face credits). The app degrades gracefully: transcripts still save and word search still works.

7.4 Dataset and training plan

Most of these dialects are not on the internet, so **the archive creates the data**: collect → AI draft → human correction → verification → export.

Open resource	What it offers	How we'd use it
Vaani (IISc Bangalore & ARTPARK, with Google)	District-wise speech across India	Closest to dialect-level speech
IndicVoices, Kathbath, Shrutilipi (AI4Bharat, IIT Madras)	Transcribed speech in 12–22 Indian languages	Adapt to related major languages first
Mozilla Common Voice	Crowd-read speech, incl. Santali, Odia, Assamese	Open training and test data
Google FLEURS, OpenSLR	Read speech in many Indian languages	Benchmarks
Meta MMS	Speech models for 1,100+ languages	Base model where Whisper has no support
Bhashini (Govt. of India), LDC-IL (CIIL Mysore)	Indian speech and text corpora, some tribal languages	Extra data (some on request)
ELAR (SOAS), PARADISEC	Linguists' field recordings of endangered languages	Real dialect audio (by permission)
Wikipedia in Tulu, Santali, Konkani, Bhojpuri…; Lingua Libre	Text and native pronunciations	Spelling, translation and pronunciation

Training steps (future work)

- Start from a multilingual base (Whisper, MMS or IndicWhisper).
- Adapt on a related major language first (e.g. Kannada before Tulu).
- Fine-tune on SwaraVeda's verified passages with lightweight adapters (**LoRA**), so even a few hours of audio helps.
- Evaluate WER on held-out verified passages before and after; better drafts mean faster correction and more data. This is a virtuous loop.

HOW IT RUNS IN PRODUCTION

8. Deployment, configuration and DevOps

Part	Hosted on	How it deploys
Frontend (static React build)	Vercel	Automatically on every merge to main; preview links for every pull request
Backend API (FastAPI + ffmpeg)	Render (web service, from render.yaml)	Automatically from the main branch; health check /health
Database + file storage	Supabase (PostgreSQL with pgvector, Storage bucket “recordings”)	Managed; tables created and migrated by the API on start-up
Source code + CI	GitHub + GitHub Actions	Every pull request runs backend tests (SQLite + real Postgres) and frontend lint + build; merged only when green

Environment settings (API keys)

All secrets live in Render's environment settings, not in code. Status as checked on the live site:

Setting	Used for	Status
DATABASE_URL	Connection to Supabase PostgreSQL	Working
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_BUCKET	Audio file storage and signed upload URLs	Working
JWT_SECRET, JWT_EXPIRE_MINUTES	Signing login tokens (7 days)	Working
GOOGLE_CLIENT_ID	Continue with Google	Working
ADMIN_EMAILS	Who becomes admin automatically	Working
CORS_ORIGINS	Which websites may call the API	Working
GROQ_API_KEY (+ GROQ_ASR_MODEL, GROQ_LLM_MODEL)	Whisper transcription and LLM translation	Working (tested live)
HF_TOKEN, EMBEDDING_MODEL	Embeddings for meaning search	Out of free credits; meaning search paused
MAX_DOWNLOAD_MB, MAX_AUDIO_MB	Upload safety limits (200 MB / 25 MB)	Defaults

HOW WE KNOW IT WORKS

9. Testing and quality

- **111 automated backend tests** (pytest) cover sign-up/login (email and phone), Google sign-in, roles, privacy rules, recordings CRUD, signed uploads, the transcription pipeline (with fake AI services), translation and the model fallback, review states, verification, search, tags, export, references and health checks.
- Tests run on SQLite for speed **and** on a real PostgreSQL in CI, so full-text search, pgvector and migrations are tested for real.
- AI services are replaced by fakes in tests, so tests are fast, free and repeatable.
- **End-to-end tests with Playwright** drive a real Chromium browser: sign up, record, upload, transcribe, correct, verify, search, the offline queue and the shelf, on desktop and phone sizes.
- **Live audit:** a script signs into the live site and visits every page on desktop and phone, failing on any JavaScript error, failed request (HTTP 4xx/5xx) or sideways scrolling.
- **Static checks:** TypeScript type-checking and oxlint on the frontend; every pull request must build.

GOOD MATERIAL FOR “WHAT WAS DIFFICULT?”

10. Challenges we faced and how we solved them

Problem	Cause	Solution
Uploads failed on the live site (500 errors)	SUPABASE_URL had been pasted with “/rest/v1/” at the end, so storage calls went to the wrong address (Supabase error PGRST125)	The API now normalises the URL (strips /rest/v1, /storage/v1, /auth/v1) and /health/db reports storage status with a clear message
Commons reference audio couldn't be transcribed (HTTP 403)	Wikimedia blocks downloads without a User-Agent	Every outgoing request identifies SwaraVeda; added a test
Translate suddenly failed	Groq retired the llama-3.3-70b-versatile model	Automatic discovery of an available chat model, with retry; added a test
Meaning search stopped	Hugging Face free credits ran out	Friendly message; exact-word search keeps working; transcripts are still saved
Offline page didn't load from cache	Responses vary by headers (Vary), so cache lookups missed	Service worker matches with ignoreVary
Clicks on the 3D shelf hit the wrong tape	Hit-testing on rotated 3D faces is unreliable in browsers	Calculated which tape was clicked from the pointer position
Logo words overlapped while changing	Old and new words animated in the same slot at full strength	Cross-fade with blur, old word leaves before the new one settles, slot width measured in advance
Recording quality	Browser noise suppression distorts singing and natural speech	Disabled it; clean on the server with ffmpeg instead
Large or long recordings	AI service limit of 25 MB per file	Compress to 32 kbps Opus mono (about 1.7 hours fits); clear message to split longer files

BE READY TO DISCUSS THESE HONESTLY

11. Limitations and future work

Current limitations

- First-draft accuracy is limited for dialects Whisper hasn't seen; human correction is required.
- Meaning search is paused until Hugging Face credits are added.
- No speaker separation yet: a conversation is one stream of segments.
- Free hosting “sleeps” when idle, so the first request after a while can take ~30–50 seconds to wake the API.
- Depends on community participation to grow the dataset.

Future work

- Fine-tune a dialect-specific speech model on the exported dataset (LoRA adapters on Whisper or MMS) and track WER per dialect.
- Speaker diarisation (who spoke when) and word-level timestamps for karaoke-style playback.
- Text-to-speech in the dialects, so learners can hear any word.
- Learning mode: flashcards and phrasebooks generated from verified passages, for younger generations.
- A map of recordings by place, and partnerships with universities, Bhashini and community organisations.
- Run embeddings locally to remove the dependency on paid credits.
- Native mobile app with background upload, and a WhatsApp voice-note submission channel.

What makes SwaraVeda different

• It focuses on **dialects with no data**, not the big languages other tools already serve.
• **Human-in-the-loop by design**: every AI output is a draft that native speakers own and approve.
• **Consent-first**: two separate consents, enforced on the server for every query and export.
• **The archive is the dataset**: every correction directly creates training data (AI draft + human truth).
• **Works offline**, where many of these communities live.
• **A cultural interface** (the dialect shelf, the multi-script logo) that invites communities in rather than feeling like a database.

PRACTISE THESE OUT LOUD

12. Viva questions and model answers

General

Q. What is the main objective of your project?

To preserve endangered Indian dialects by making it easy for communities to record speech, get an AI first-draft transcript and translation, correct and verify it, and turn the result into a searchable archive and an open, consent-based training dataset.

Q. Why did you choose this problem?

Hundreds of Indian dialects are spoken by small, ageing communities and have almost no written or digital record. Mainstream speech technology ignores them because there's no training data. Preserving them now matters both culturally and for future technology.

Q. Who are the users?

Contributors (community members who record and correct), verifiers (trusted native speakers or linguists who approve) and admins (who manage roles and dialects). Researchers use the search and the dataset export.

Q. What is new in your approach compared to existing apps?

Existing tools either just store audio or offer AI only for major languages. We combine AI drafting with community correction, per-recording consent, offline collection and dataset export, so the archive itself becomes training data for unsupported dialects.

Q. What is the meaning of SwaraVeda and Vani?

Swara means voice or musical note and Veda means knowledge in Sanskrit: “knowledge of voices”. Vani means speech; it's the name of our AI assistant.

Architecture & technology

Q. Explain your architecture.

Three tiers: a React single-page app in the browser; a FastAPI REST backend on Render; and Supabase for PostgreSQL and file storage. AI runs through APIs: Groq for Whisper and the LLM, Hugging Face for embeddings. Audio goes straight from the browser to storage using signed URLs, so it doesn't pass through our server.

Q. Why FastAPI instead of Django or Flask?

FastAPI validates requests automatically with type hints (Pydantic), generates interactive API docs, supports background tasks and is very fast. Django is heavier than we needed for a pure API; Flask would need many add-ons for validation and docs.

Q. Why React?

The app is highly interactive: live recording, a waveform player synced with the transcript, a 3D shelf and live job progress. React's component and state model handles this cleanly, and TypeScript catches errors early.

Q. Why PostgreSQL and not MongoDB?

Our data is relational (dialects → recordings → segments, users, tags), and we need joins, constraints and transactions. PostgreSQL also gives us full-text search and, with pgvector, vector search in one database.

Q. What is a REST API?

An interface where resources (recordings, segments, dialects) have URLs and are acted on with HTTP methods: GET reads, POST creates, PATCH updates, DELETE removes. Data is exchanged as JSON, and each request carries its own authentication (stateless).

Q. What is an ORM? Which one did you use?

An Object-Relational Mapper lets us work with database rows as Python objects. We used SQLModel, which combines SQLAlchemy (database) and Pydantic (validation), so one class defines both the table and the API schema.

Q. Why upload directly to storage instead of through the API?

Audio files can be large. Signed URLs let the browser upload straight to Supabase Storage, which saves server bandwidth and memory, is faster, and avoids request-size limits on the free server. The URL is short-lived and limited to the user's folder.

Q. How do the frontend and backend communicate?

Over HTTPS using JSON with Axios. Every request carries a JWT in the Authorization header. CORS settings ensure only our site's origin can call the API from a browser.

Authentication & security

Q. How does login work?

The password is checked against a bcrypt hash; on success the server returns a JWT signed with a secret. The browser sends it with each request; the server verifies the signature and expiry and loads the user. No session is stored on the server.

Q. What is a JWT?

A JSON Web Token: a header, a payload (user ID, expiry) and a signature, Base64-encoded and joined by dots. Anyone can read the payload, but only the server can create a valid signature, so it can't be forged.

Q. Why bcrypt for passwords?

It is a slow, salted hashing algorithm. Salting means identical passwords get different hashes, and slowness makes brute-force guessing very expensive. We never store or log plain passwords.

Q. How does Google sign-in work?

Google's button returns a signed ID token. Our server verifies its RS256 signature with Google's public keys, checks that the audience is our client ID and the issuer is Google, then finds or creates the user and issues our own JWT.

Q. How do you implement roles?

Each user has a role: contributor, verifier or admin. FastAPI dependencies (require_verifier, require_admin) check the role on the server before sensitive actions such as verifying, exporting or changing roles. The UI hides buttons too, but the server is the real gate.

Q. How do you protect private recordings?

A single rule (visible_to) is applied to every query: lists, detail, segments, search, stats. Non-public recordings are visible only to their uploader, verifiers and admins; others get 404, so even their existence isn't revealed.

Q. Where are API keys stored?

In the hosting provider's (Render's) environment variables, read by pydantic-settings at start-up. They are never committed to GitHub or sent to the browser.

AI / NLP

Q. Are you using NLP? Where?

Yes: automatic speech recognition (Whisper), machine translation and transliteration (an LLM), semantic search with sentence embeddings, and full-text information retrieval. Audio cleaning is signal processing, not NLP.

Q. What is Whisper and why did you use it?

An open speech recognition model from OpenAI, a Transformer encoder–decoder trained on a huge amount of multilingual audio. It supports around 100 languages, gives timestamps and is robust to noise and accents. We use large-v3, its most accurate version, via Groq for speed.

Q. Whisper doesn't support Tulu or Toda. How do you transcribe them?

Each dialect stores the closest language Whisper knows (Tulu → Kannada, Toda → Tamil, Bhojpuri → Hindi), passed as a hint. That gives a usable first draft in the right script, which native speakers then correct. Those corrections become the data to fine-tune a dialect-specific model later.

Q. Why preprocess audio?

Whisper works at 16 kHz mono. Our high-pass filter removes low rumble, loudness normalisation makes quiet speakers clear, and Opus compression keeps files under the 25 MB API limit while preserving speech quality.

Q. What is semantic search and how does it work?

Searching by meaning instead of exact words. Each passage is converted into a 384-dimension vector by a multilingual sentence-embedding model; the query is converted the same way; pgvector returns passages whose vectors are closest by cosine distance.

Q. What is cosine similarity?

A measure of the angle between two vectors: the dot product divided by the product of their lengths. 1 means the same direction (same meaning), 0 unrelated. pgvector's <=> operator returns cosine distance, which is 1 minus the similarity.

Q. How is full-text search different?

It matches actual words using PostgreSQL's to_tsvector and tsquery with an index, ranked by ts_rank. We use the “simple” configuration so Indian-language words aren't damaged by English stemming.

Q. How does translation work?

We send the passage to an LLM on Groq with a prompt saying it may be a dialect of a given language and may contain transcription errors, and ask for JSON with an English translation and a transliteration. The result is a suggestion the user can edit.

Q. What happens if the AI model is retired?

That actually happened. The API now detects a “model does not exist” error, asks Groq for the list of available models, picks the best chat model, remembers it and retries, so translation keeps working without code changes.

Q. How will you measure accuracy?

Word Error Rate: substitutions plus deletions plus insertions, divided by the number of words in the correct transcript. Because we store both the AI draft and the human correction for each segment, we can compute WER per dialect, before and after fine-tuning.

Q. Are you training your own model?

Not yet. The system currently uses pretrained models. The dataset export and training plan are ready: start from Whisper or MMS, adapt on a related language, then fine-tune with LoRA on verified passages.

Q. What is LoRA?

Low-Rank Adaptation: instead of retraining all of a large model's weights, you train small extra matrices added to some layers. It needs far less data and compute, which suits low-resource dialects.

Q. What datasets could you use?

Our own verified, consented passages first; plus open resources such as Vaani (IISc/ARTPARK), AI4Bharat's IndicVoices and Kathbath, Mozilla Common Voice, Google FLEURS, Meta MMS, Bhashini, and field archives such as ELAR and PARADISEC (with permission).

Features & workflow

Q. Walk us through recording to verified transcript.

Record in the wizard with consent → signed upload to storage → transcription job: download, ffmpeg cleaning, Whisper with the dialect hint → segments with timestamps saved and embedded → a native speaker corrects each segment while listening → a verifier approves → the recording becomes verified, searchable and (with consent) exportable.

Q. How does offline recording work?

If the upload fails for lack of connection, the audio and details are saved in IndexedDB on the device. When the browser comes back online, the queue is uploaded in order. A service worker caches the app so it opens offline.

Q. Why run transcription as a background job?

Transcribing can take from seconds to minutes. A background task lets the API reply immediately with a job ID, and the page polls the job's progress, so the user isn't blocked and timeouts are avoided.

Q. How does the transcript stay in sync with the audio?

Each segment stores start_ms and end_ms. As the audio plays, the current time is compared with the segment ranges to highlight the one being spoken; clicking a segment seeks the audio to its start.

Q. What is in the exported dataset?

For each verified, AI-consented segment: audio URL, start and end times, the AI draft, the human correction, transliteration, translation, dialect, speaker label and licence, as JSONL or CSV.

Q. How did you build the dialect shelf?

Each dialect is a VHS-style tape drawn with CSS 3D transforms and generated canvas artwork. Hover pulls it out; click opens a detail panel with info, its recordings (Voices), a Record button and links to hear the dialect elsewhere.

Q. How does the logo change languages smoothly?

A React hook advances an index every second. The ReactiveLogo component measures each word's width in advance, cross-fades the old and new words with the Web Animations API (opacity, blur, small movement) and changes only that word's colour.

Testing, deployment & practical

Q. How did you test the project?

111 pytest tests on the backend (with SQLite and real PostgreSQL in CI), fake AI services for repeatable tests, Playwright end-to-end tests in a real browser, and a live audit of every page on desktop and phone sizes. GitHub Actions runs tests on every change.

Q. How is it deployed?

The frontend on Vercel and the API on Render, both deploying automatically from the main branch after tests pass; Supabase hosts the database and audio storage.

Q. What was the hardest bug?

Live uploads failing with a 500 error. We traced it to the Supabase URL having been pasted with “/rest/v1/”, which made storage calls go to the wrong endpoint. We fixed it by normalising the URL and adding a health check that reports storage status.

Q. How does the app scale?

The API is stateless, so more instances can be added; uploads bypass the server; heavy AI runs on external GPU services; and the database uses indexes for search and recording lookups. Background work could move to a dedicated job queue (e.g. Celery or Redis) at higher load.

Q. What would you improve with more time?

Fine-tune a dialect-specific ASR model with the collected data, add speaker diarisation, move embeddings in-house, and build learning features for the younger generation.

Q. What are the ethical considerations?

Informed consent per recording (listening and AI training separately), privacy by default, crediting speakers and licences, community ownership of corrections, and being transparent that AI output is only a draft.

TERMS YOU MAY BE ASKED TO DEFINE

13. Glossary

Term	Meaning
ASR	Automatic Speech Recognition: converting speech audio to text
NLP	Natural Language Processing: computers understanding and generating human language
Transformer	Neural network architecture based on attention; used by Whisper and LLMs
LLM	Large Language Model: a Transformer trained on huge text, able to translate and follow instructions
Embedding	A list of numbers representing meaning; similar meanings give nearby vectors
pgvector	PostgreSQL extension for storing vectors and finding nearest neighbours
Cosine distance	1 − cosine similarity; how far apart two vectors point
Full-text search	Word-based search using an index of normalised tokens (tsvector/tsquery)
WER	Word Error Rate = (S + D + I) / N; the standard ASR accuracy measure
LoRA	Low-Rank Adaptation, an efficient way to fine-tune large models with little data
JWT	JSON Web Token, a signed, self-contained login token
bcrypt	Slow, salted password-hashing algorithm
CORS	Cross-Origin Resource Sharing; browser rule controlling which sites may call an API
Signed URL	A temporary URL granting permission to upload or download one file
IndexedDB	A database built into the browser for storing data (and files) on the device
Service worker	A background browser script that can cache files so a site works offline
MediaRecorder	Browser API that records audio or video from the microphone or camera
ISO 639-3	Three-letter language codes (e.g. tcy for Tulu)
E.164	International phone number format, e.g. +919876543210
LUFS	Loudness Units relative to Full Scale; used for loudness normalisation
Opus	Modern, efficient audio codec, excellent for speech at low bitrates
CI/CD	Continuous Integration / Deployment: automatic testing and releasing of every change
Human-in-the-loop	AI produces drafts; humans review, correct and approve